

Contexte

On analyse la télémétrie des serveurs chez un fournisseur de Cloud pour comprendre les goulots d'étranglement et prédire les temps de réponse.

Objectifs :

- Générer un dataset synthétique réaliste (Data Engineering).
- Nettoyer des données bruitées (valeurs manquantes et outliers).
- Analyser les corrélations (EDA).
- Modéliser une relation simple par régression linéaire (Introduction ML).

Exercice 1. Génération et Nettoyage (Data Engineering)

1. **Simulation des données :** Générez un DataFrame de $N = 5000$ lignes avec les colonnes suivantes. Assurez-vous d'utiliser une graine aléatoire (`random_state`) pour la reproductibilité.

- (a) **CPU_Usage :** Variable aléatoire suivant une loi Uniforme $\mathcal{U}[10,90]$.
- (b) **Memory :** $0.5 \times \text{CPU_Usage} + \epsilon$, où $\epsilon \sim \mathcal{N}(0,5)$ (Bruit Gaussien).
- (c) **Response_Time (ms) :** On souhaite une relation linéaire pour la modélisation future.

$$\text{Response_Time} = 2 \times \text{CPU_Usage} + 50 + \text{Bruit}$$

Le bruit suivra une loi Exponentielle (`scale=10`) pour simuler des latences réseaux occasionnelles.

- (d) **Error_Rate :** Variable binaire (0 ou 1). La probabilité d'erreur dépend de la charge :
 - Si $\text{CPU_Usage} > 80$, alors $P(\text{Error} = 1) = 0.30$.
 - Sinon, $P(\text{Error} = 1) = 0.01$.

2. Injection de défauts et Nettoyage

- (a) **Injection :** Introduisez artificiellement des valeurs `NaN` dans 5% des cellules de la colonne `Memory` et des outliers aberrants (ex: 9999) dans 1% des lignes de `Response_Time`.
- (b) **Imputation :** Détectez les valeurs manquantes et remplacez-les par la **médiane** (plus robuste que la moyenne).
- (c) **Filtrage :** Filtrez les outliers de la colonne `Response_Time` en utilisant la méthode de l'écart interquartile (IQR).

Exercice 2. Analyse Univariée et Multivariée

1. **Distributions :** Tracez les histogrammes de chaque variable quantitative avec la courbe de densité estimée (KDE).
2. **Matrice de Corrélation :** Calculez la matrice de corrélation de Pearson et affichez-la (Heatmap).
3. **Analyse :** Pourquoi la corrélation entre CPU et Memory est-elle forte ? S'agit-il d'une causalité dans notre simulation ?
4. **Pairplot :** Utilisez `sns.pairplot` en colorant les points selon `Error_Rate`. Que remarquez-vous sur la séparabilité des classes (erreurs vs fonctionnement normal) ?

Exercice 3. Régression Linéaire (Comparaison Théorie vs Pratique) On cherche à prédire le **temps de réponse** (Y) en fonction du **CPU** (X) suivant un modèle : $Y = aX + b$.

1. **Approche Scikit-Learn :** Utilisez `LinearRegression` pour ajuster le modèle sur vos données nettoyées. Affichez les coefficients.
2. **Approche Mathématique :** Calculez les coefficients "à la main" via les formules des moindres carrés ordinaires :

$$a = \frac{Cov(X,Y)}{Var(X)} \quad \text{et} \quad b = \bar{Y} - a\bar{X}$$

Attention : Par défaut, `numpy.cov` calcule la covariance non biaisée (divisée par $n - 1$) alors que la variance `numpy.var` est souvent divisée par n . Assurez-vous d'utiliser la même convention (paramètre `ddof`) pour obtenir un résultat cohérent.

3. **Conclusion :** Comparez les valeurs obtenues par les deux méthodes. Calculez le score R^2 .